ZRC-20: Zoo Fungible Token Standard with Conservation Hooks

Antje Worring & Zach Kelling

Zoo Labs Foundation

January 2025

Abstract

We present ZRC-20, the canonical fungible token standard for the Zoo L2 chain (Chain ID: 200200). ZRC-20 is fully ERC-20 compatible and extends the Lux LRC-20 standard (LP-3020) with three conservation-specific mechanisms: (1) auto-donation on transfer, where a configurable fraction of each transfer (up to 5%) is directed to a verified conservation fund; (2) burn-for-conservation, where token holders permanently burn tokens in exchange for soulbound impact credits priced by the Zoo Impact Oracle; and (3) on-chain impact attribution, where cumulative conservation contributions are tracked per address as non-transferable records. The standard also incorporates governance-ready voting (EIP-5805), gasless approvals (EIP-2612), flash minting (EIP-3156), and cross-chain bridgeability (LP-3800). We provide the complete Solidity interface, a reference implementation, and a formal analysis of the donation mechanism's economic properties.

Keywords: fungible token, ERC-20, conservation, auto-donation, impact credits, DeFi

1 Introduction

Every ERC-20 transfer is an economic event. In the Zoo ecosystem, every economic event should generate measurable conservation impact. The challenge is achieving this without breaking the composability that makes ERC-20 the most widely adopted token standard.

ZRC-20 solves this by adding conservation hooks that activate during standard `transfer` and `transferFrom` calls. These hooks are transparent to existing DeFi protocols: a Uniswap pool, a lending protocol, or a bridge contract that interacts with a ZRC-20 token sees a standard ERC-20 interface. The conservation extensions are additive—they use separate function selectors and emit dedicated events.

1.1 Design Goals

1. **Drop-in ERC-20**: Every wallet, DEX, aggregator, and bridge that speaks ERC-20 works with ZRC-20 without modification.
2. **Conservation alignment**: Value flows toward conservation automatically, without requiring user action.
3. **Transparent impact**: Donation amounts and impact credits are on-chain, auditable, and attributed to specific addresses.
4. **Governance readiness**: Voting weight and delegation are first-class features for DAO-governed conservation funding.
5. **Capital efficiency**: Flash minting and gasless approvals reduce friction for DeFi composability.

1.2 Contributions

1. The complete IZRC20 interface specification with conservation extensions (Section 3).
2. A reference implementation with auto-donation, burn-for-conservation, and impact tracking (Section 4).
3. Formal analysis of the donation mechanism's economic equilibrium (Section 6).

4. Extension interfaces for burnable, mintable, capped, voting, permit, flash mint, and bridgeable variants (Section 5).

5. Security analysis covering reentrancy, oracle manipulation, and governance capture (Section 8).

2 Background

2.1 ERC-20

ERC-20 [1] defines the standard fungible token interface: `transfer`, `transferFrom`, `approve`, `allowance`, `balanceOf`, `totalSupply`. It is implemented by over 500,000 contracts on Ethereum mainnet and supported by every major wallet, exchange, and DeFi protocol.

2.2 Transfer Taxes and Fee-on-Transfer

Several tokens implement fee-on-transfer mechanisms (e.g., SafeMoon, REFLECT). These modify the received amount during `transfer`, breaking DeFi protocols that assume `amount_sent == amount_received`. ZRC-20 avoids this pattern. The auto-donation is a separate transfer to the conservation fund, not a deduction from the recipient's amount.

2.3 Lux LRC-20

LRC-20 (LP-3020) [5] is the Lux Network's extension of ERC-20 with a modular extension suite: burnable, mintable, capped, votes, permit, flash mint, and bridgeable. ZRC-20 inherits the full LRC-20 extension set and adds conservation-specific functionality on top.

3 Interface Specification

3.1 Core Interface

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

interface IZRC20 {
    // ERC-20 Core (unchanged)
    function name() external view returns (string memory);
    function symbol() external view returns (string memory);
    function decimals() external view returns (uint8);
    function totalSupply() external view returns (uint256);
    function balanceOf(address account)
        external view returns (uint256);
    function transfer(address to, uint256 amount)
        external returns (bool);
    function allowance(address owner, address spender)
        external view returns (uint256);
    function approve(address spender, uint256 amount)
        external returns (bool);
    function transferFrom(
        address from, address to, uint256 amount
    ) external returns (bool);

    event Transfer(
```

```
        address indexed from,
        address indexed to,
        uint256 value
    );
    event Approval(
        address indexed owner,
        address indexed spender,
        uint256 value
    );

    // Conservation Extensions
    function conservationFund()
        external view returns (address);
    function donationBasisPoints()
        external view returns (uint16);
    function totalImpactCredits()
        external view returns (uint256);
    function impactCreditsOf(address account)
        external view returns (uint256);
    function burnForConservation(uint256 amount)
        external returns (uint256 creditsIssued);

    event ConservationDonation(
        address indexed from,
        address indexed to,
        uint256 transferAmount,
        uint256 donationAmount
    );
    event BurnedForConservation(
        address indexed burner,
        uint256 amountBurned,
        uint256 creditsIssued
    );
    event DonationRateUpdated(
        uint16 oldRate,
        uint16 newRate
    );
}
```

Listing 1: IZRC20 core interface

3.2 Transfer Semantics

On every `transfer(to, amount)` call where `donationBasisPoints > 0`:

1. Compute `donationAmount = amount * donationBasisPoints / 10000`.
2. Transfer `donationAmount` to `conservationFund()`.
3. Transfer `amount` to `to` (the recipient receives the full amount).
4. The sender's balance decreases by `amount + donationAmount`.
5. Emit `ConservationDonation(from, to, amount, donationAmount)`.
6. Emit standard `Transfer` events for both the recipient and the fund.

The recipient always receives the stated `amount`. The donation is an additional charge to the sender. This preserves the invariant that downstream contracts receive the amount they expect.

Definition 3.1 (Sender Cost)**.** *For a transfer of amount a with donation rate d basis points:*

$$C_{sender} = a + \left\lfloor \frac{a \cdot d}{10000} \right\rfloor \tag{1}$$

The sender must have balance $\geq C_{sender}$ for the transfer to succeed.

4 Reference Implementation

```
contract ZRC20 is ERC20, IZRC20 {
    address public conservationFund;
    uint16 public donationBasisPoints;
    address public impactOracle;

    mapping(address => uint256) private _impactCredits;
    uint256 private _totalImpactCredits;

    uint16 public constant MAX_DONATION_BPS = 500;

    constructor(
        string memory name_,
        string memory symbol_,
        address fund_,
        uint16 donationBps_,
        address oracle_
    ) ERC20(name_, symbol_) {
        require(donationBps_ <= MAX_DONATION_BPS,
            "rate␣exceeds␣5%");
        conservationFund = fund_;
        donationBasisPoints = donationBps_;
        impactOracle = oracle_;
    }

    function _update(
        address from,
        address to,
        uint256 amount
    ) internal override {
        if (from != address(0) && to != address(0)
            && donationBasisPoints > 0
            && to != conservationFund) {
            uint256 donation = (amount
                * donationBasisPoints) / 10000;
            if (donation > 0) {
                super._update(from,
                    conservationFund, donation);
                emit ConservationDonation(
                    from, to, amount, donation);
            }
        }
        super._update(from, to, amount);
    }

    function burnForConservation(uint256 amount)
```

```
        external returns (uint256 creditsIssued) {
        _burn(msg.sender, amount);
        creditsIssued = IImpactOracle(impactOracle)
            .getCredits(amount);
        _impactCredits[msg.sender] += creditsIssued;
        _totalImpactCredits += creditsIssued;
        emit BurnedForConservation(
            msg.sender, amount, creditsIssued);
    }

    function totalImpactCredits()
        external view returns (uint256) {
        return _totalImpactCredits;
    }

    function impactCreditsOf(address account)
        external view returns (uint256) {
        return _impactCredits[account];
    }
}
```

Listing 2: ZRC20 reference implementation (core transfer logic)

4.1 Impact Oracle Interface

```
interface IImpactOracle {
    function getCredits(uint256 tokenAmount)
        external view returns (uint256 credits);
    function creditPrice()
        external view returns (uint256 tokensPerCredit);
}
```

Listing 3: IImpactOracle interface

The oracle returns the number of conservation credits for a given token burn amount. The credit price is based on verified conservation project costs (e.g., cost per hectare of habitat protected, cost per animal tracked). The oracle is registered in the Zoo Contract Registry (ZIP-0100) and can be updated by governance.

5 Extension Interfaces

ZRC-20 tokens MAY implement any combination of the following extensions, inherited from the LRC-20 suite:

5.1 Burnable

```
interface IZRC20Burnable is IZRC20 {
    function burn(uint256 amount) external;
    function burnFrom(address account, uint256 amount)
        external;
}
```

Listing 4: IZRC20Burnable

5.2 Mintable

```
interface IZRC20Mintable is IZRC20 {
    function mint(address to, uint256 amount) external;
    event Minted(
        address indexed to,
        uint256 amount,
        address indexed minter
    );
}
```

Listing 5: IZRC20Mintable

5.3 Votes (EIP-5805 Compatible)

```
interface IZRC20Votes is IZRC20 {
    function getVotes(address account)
        external view returns (uint256);
    function getPastVotes(
        address account, uint256 timepoint
    ) external view returns (uint256);
    function getPastTotalSupply(uint256 timepoint)
        external view returns (uint256);
    function delegates(address account)
        external view returns (address);
    function delegate(address delegatee) external;
    function delegateBySig(
        address delegatee, uint256 nonce,
        uint256 expiry,
        uint8 v, bytes32 r, bytes32 s
    ) external;
}
```

Listing 6: IZRC20Votes

5.4 Permit (EIP-2612)

```
interface IZRC20Permit is IZRC20 {
    function permit(
        address owner, address spender,
        uint256 value, uint256 deadline,
        uint8 v, bytes32 r, bytes32 s
    ) external;
    function nonces(address owner)
        external view returns (uint256);
    function DOMAIN_SEPARATOR()
        external view returns (bytes32);
}
```

Listing 7: IZRC20Permit

5.5 Flash Mint (EIP-3156)

```
interface IZRC20FlashMint is IZRC20 {
    function maxFlashLoan(address token)
        external view returns (uint256);
    function flashFee(address token, uint256 amount)
        external view returns (uint256);
    function flashLoan(
        address receiver, address token,
        uint256 amount, bytes calldata data
    ) external returns (bool);
}
```

Listing 8: IZRC20FlashMint

5.6 Bridgeable (LP-3800)

```
interface IZRC20Bridgeable is IZRC20 {
    function bridge()
        external view returns (address);
    function mintFromBridge(
        address to, uint256 amount,
        bytes32 sourceChainTxHash
    ) external;
    function burnForBridge(
        uint256 amount,
        bytes32 destinationChainId
    ) external;
    event BridgeMint(
        address indexed to, uint256 amount,
        bytes32 indexed sourceTxHash
    );
    event BridgeBurn(
        address indexed from, uint256 amount,
        bytes32 indexed destChainId
    );
}
```

Listing 9: IZRC20Bridgeable

6 Economic Analysis

6.1 Auto-Donation Equilibrium

Let d be the donation rate (basis points), V the daily transfer volume, and F the conservation fund balance. The daily conservation inflow is:

$$I_{\text{daily}} = V \cdot \frac{d}{10000} \tag{2}$$

For the ZOO token with $d = 50$ (0.5%) and estimated daily volume $V = 10{,}000{,}000$ ZOO:

$$I_{\text{daily}} = 10{,}000{,}000 \cdot 0.005 = 50{,}000 \text{ ZOO/day} \tag{3}$$

Field	Type	Description
standard	string	`"ZRC-20"`
extensions	string[]	Implemented extension names
conservationFund	address	Fund receiving auto-donations
donationBasisPoints	uint16	Current donation rate
impactOracleAddress	address	Oracle for credit pricing

Table 1: ZRC-20 token registration metadata.

6.2 Burn-for-Conservation Deflation

Each `burnForConservation` call reduces total supply permanently. If fraction ϕ of holders burn annually:

$$S_{t+1} = S_t \cdot (1 - \phi) \tag{4}$$

This creates deflationary pressure proportional to conservation engagement. The impact credits serve as a non-financial reward that preserves the conservation signal without creating a speculative secondary market.

Theorem 6.1 (Supply Floor)**.** *Under the ZRC-20 mechanism, total supply is bounded below by:*

$$S_{\min} = S_0 \cdot (1 - \phi)^t \tag{5}$$

As $t \to \infty$, $S_{\min} \to 0$. In practice, burn rate decreases as remaining supply decreases (fewer tokens to burn), establishing an equilibrium where supply stabilizes.

6.3 Donation Rate Cap

The 5% (500 basis points) cap on donation rate prevents governance capture:

Proposition 6.2 (Anti-Extraction Bound)**.** *If the donation rate $d \leq 500$ basis points, the maximum value extracted from any single transfer is 5% of the transfer amount. This bounds the cost of DeFi composability: a multi-hop trade through 3 pools loses at most $1 - 0.95^3 = 14.3\%$ to conservation donations.*

In practice, most ZRC-20 tokens use $d = 50$ (0.5%), making the three-hop cost $1 - 0.995^3 = 1.5\%$.

7 Token Registration

All ZRC-20 tokens deployed on Zoo L2 MUST register in the Zoo Contract Registry (ZIP-0100) with:

8 Security Analysis

8.1 Reentrancy

The `burnForConservation` function interacts with an external oracle. The reference implementation follows checks-effects-interactions: the burn (state change) occurs before the oracle call (external interaction). Implementations MUST use a reentrancy guard.

8.2 Oracle Manipulation

A flash loan could manipulate the oracle's credit price within a single transaction. Mitigations:

- The oracle uses time-weighted average pricing (TWAP) over 24 hours, not spot price.
- Multi-source aggregation from multiple data providers.
- A per-block burn cap prevents large single-transaction burns.

8.3 Donation Rate Governance

Changes to `donationBasisPoints` go through a timelock (minimum 48 hours) to prevent flash governance attacks. The rate MUST NOT exceed 500 basis points.

8.4 Rounding

Donation amount calculation rounds down: `(amount * bps) / 10000`. For small transfers where the donation rounds to zero, no donation is made. This prevents dust accumulation and ensures transfers of 1 wei always succeed.

8.5 Bridge Security

`mintFromBridge` is callable only by the registered bridge contract. The source chain transaction hash is tracked to prevent replay.

9 Related Work

ERC-20 [1] defines the base fungible token standard. EIP-2612 [2] adds gasless approvals via permit. EIP-3156 [3] standardizes flash loans. EIP-5805 [4] defines the voting interface. SafeMoon [12] and similar tokens implement fee-on-transfer, but break DeFi composability by modifying the received amount. ZRC-20 avoids this by charging the sender, not reducing the recipient amount.

Toucan Protocol [13] tokenizes carbon credits as ERC-20 tokens. ZRC-20 differs by integrating conservation impact at the token standard level rather than wrapping external credits. KlimaDAO [14] uses token burns to retire carbon credits; ZRC-20's burn-for-conservation mechanism is analogous but targets biodiversity conservation rather than carbon.

10 Conclusion

ZRC-20 extends the most widely adopted token standard with conservation alignment while preserving full ERC-20 compatibility. The auto-donation mechanism generates sustained conservation funding from organic economic activity. The burn-for-conservation mechanism creates deflationary pressure tied to conservation engagement. The soulbound impact credits provide verifiable proof of contribution without creating speculative markets.

The standard's modular extension system—inherited from LRC-20 and extended with conservation hooks—enables token creators to compose exactly the feature set needed for their conservation use case, from simple donation tokens to governance-ready voting tokens with cross-chain bridgeability.

Acknowledgments

This work was supported by Zoo Labs Foundation. We thank the OpenZeppelin team for the ERC-20 implementation that serves as the foundation for ZRC-20, and the Lux team for the LRC-20 extension suite.

References

[1] F. Vogelsteller and V. Buterin, "EIP-20: Token Standard," Ethereum Improvement Proposals, 2015.

[2] M. Lundfall, "EIP-2612: Permit—712-Signed Approvals," Ethereum Improvement Proposals, 2020.

[3] A. Martinez and M. San Juan, "EIP-3156: Flash Loans," Ethereum Improvement Proposals, 2020.

[4] R. Solow, "EIP-5805: Voting with Delegation," Ethereum Improvement Proposals, 2022.

[5] Lux Partners, "LP-3020: LRC-20 Fungible Token Standard," Lux Protocol Standards, 2022.

[6] Lux Partners, "LP-3800: Bridged Asset Standard," Lux Protocol Standards, 2022.

[7] Lux Partners, "LP-6000: B-Chain Bridge Specification," Lux Protocol Standards, 2022.

[8] Zoo Labs Foundation, "ZIP-0100: Zoo Contract Registry," Zoo Improvement Proposals, 2024.

[9] Zoo Labs Foundation, "ZIP-0500: ESG Principles for Conservation Impact," Zoo Improvement Proposals, 2024.

[10] Zoo Labs Foundation, "ZIP-0501: Conservation Impact Measurement," Zoo Improvement Proposals, 2024.

[11] OpenZeppelin, "OpenZeppelin Contracts: Secure Smart Contract Library," `https://openzeppelin.com/contracts`, 2023.

[12] SafeMoon, "SafeMoon Protocol: Tokenomics with Reflection," Technical Documentation, 2021.

[13] Toucan Protocol, "Toucan: Infrastructure for Tokenized Carbon Credits," Technical Documentation, 2022.

[14] KlimaDAO, "KlimaDAO: A Carbon-Backed Currency," Technical Documentation, 2021.

[15] H. Adams et al., "Uniswap v2 Core," Technical Documentation, 2020.

[16] R. Leshner and G. Hayes, "Compound: The Money Market Protocol," Technical Documentation, 2019.

[17] C. Reitwießner et al., "EIP-165: Standard Interface Detection," Ethereum Improvement Proposals, 2018.

[18] Zoo Labs Foundation, "ZIP-0015: Zoo L2 Chain Architecture," Zoo Improvement Proposals, 2024.

[19] J. Transmissions11 et al., "EIP-4626: Tokenized Vault Standard," Ethereum Improvement Proposals, 2022.

[20] Zoo Labs Foundation, "ZIP-0013: LP Standards Conformance," Zoo Improvement Proposals, 2024.